

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```
df=pd.read_csv('car_data.csv')
```

```
df.head()
```

	Car_Name	Year	Selling_Price	Present_Price	Driven_kms	Fuel_Type	Selling_type	Transmission	Owner	
0	ritz	2014	3.35	5.59	27000	Petrol	Dealer	Manual	0	
1	sx4	2013	4.75	9.54	43000	Diesel	Dealer	Manual	0	
2	ciaz	2017	7.25	9.85	6900	Petrol	Dealer	Manual	0	
3	wagon r	2011	2.85	4.15	5200	Petrol	Dealer	Manual	0	
4	swift	2014	4.60	6.87	42450	Diesel	Dealer	Manual	0	

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
df.tail()
```

	Car_Name	Year	Selling_Price	Present_Price	Driven_kms	Fuel_Type	Selling_type	Transmission	Owner	
296	city	2016	9.50	11.6	33988	Diesel	Dealer	Manual	0	
297	brio	2015	4.00	5.9	60000	Petrol	Dealer	Manual	0	
298	city	2009	3.35	11.0	87934	Petrol	Dealer	Manual	0	
299	city	2017	11.50	12.5	9000	Diesel	Dealer	Manual	0	
300	brio	2016	5.30	5.9	5464	Petrol	Dealer	Manual	0	

```
df.shape
```

```
(301, 9)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 301 entries, 0 to 300
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Car_Name        301 non-null   object
1   Year            301 non-null   int64
2   Selling_Price   301 non-null   float64
3   Present_Price   301 non-null   float64
4   Driven_kms      301 non-null   int64
5   Fuel_Type       301 non-null   object
6   Selling_type    301 non-null   object
7   Transmission    301 non-null   object
8   Owner           301 non-null   int64
dtypes: float64(2), int64(3), object(4)
memory usage: 21.3+ KB
```

```
df.describe()
```

	Year	Selling_Price	Present_Price	Driven_kms	Owner
count	301.000000	301.000000	301.000000	301.000000	301.000000
mean	2013.627907	4.661296	7.628472	36947.205980	0.043189
std	2.891554	5.082812	8.642584	38886.883882	0.247915
min	2003.000000	0.100000	0.320000	500.000000	0.000000
25%	2012.000000	0.900000	1.200000	15000.000000	0.000000
50%	2014.000000	3.600000	6.400000	32000.000000	0.000000
75%	2016.000000	6.000000	9.900000	48767.000000	0.000000
max	2018.000000	35.000000	92.600000	500000.000000	3.000000

```
df.isnull().sum()
```

```

      0
Car_Name  0
Year      0
Selling_Price  0
Present_Price  0
Driven_kms  0
Fuel_Type  0
Selling_type  0
Transmission  0
Owner       0

```

```
dtype: int64
```

df

	Car_Name	Year	Selling_Price	Present_Price	Driven_kms	Fuel_Type	Selling_type	Transmission	Owner	
0	ritz	2014	3.35	5.59	27000	Petrol	Dealer	Manual	0	
1	sx4	2013	4.75	9.54	43000	Diesel	Dealer	Manual	0	
2	ciaz	2017	7.25	9.85	6900	Petrol	Dealer	Manual	0	
3	wagon r	2011	2.85	4.15	5200	Petrol	Dealer	Manual	0	
4	swift	2014	4.60	6.87	42450	Diesel	Dealer	Manual	0	
...	...	...	...	...	...	...	...	...	...	
296	city	2016	9.50	11.60	33988	Diesel	Dealer	Manual	0	
297	brio	2015	4.00	5.90	60000	Petrol	Dealer	Manual	0	
298	city	2009	3.35	11.00	87934	Petrol	Dealer	Manual	0	
299	city	2017	11.50	12.50	9000	Diesel	Dealer	Manual	0	
300	brio	2016	5.30	5.90	5464	Petrol	Dealer	Manual	0	

301 rows × 9 columns

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
df=pd.read_csv('car_data.csv')
x=df[['Car_Name', 'Year', 'Selling_Price', 'Present_Price', 'Driven_kms', 'Fuel_Type', 'Selling_type', 'Transmission']]
y=df['Owner']
```

```
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
```

```
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
```

```
categorical_features = ['Car_Name', 'Fuel_Type', 'Selling_type', 'Transmission']  
one_hot = OneHotEncoder(handle_unknown='ignore') # Added handle_unknown='ignore'  
transformer = ColumnTransformer([('one_hot', one_hot, categorical_features)], remainder='passthrough')
```

```
x_train_transformed = transformer.fit_transform(x_train)
```

```
model = LinearRegression()  
model.fit(x_train_transformed, y_train)
```

```
print("Model fitted successfully after one-hot encoding categorical features.")
```

Model fitted successfully after one-hot encoding categorical features.

```
x_test_transformed = transformer.transform(x_test)  
y_pred = model.predict(x_test_transformed)  
mse = mean_squared_error(y_test, y_pred)  
r2 = r2_score(y_test, y_pred)  
print(mse)  
print(r2)
```

```
0.01131659668557271  
0.2981823955497328
```

```
new_data = {  
    'Car_Name': ['ritz'],  
    'Year': [2015],  
    'Selling_Price': [4.0],  
    'Present_Price': [5.5],  
    'Driven_kms': [25000],  
    'Fuel_Type': ['Petrol'],  
    'Selling_type': ['Dealer'],  
    'Transmission': ['Manual']  
}  
new = pd.DataFrame(new_data)  
display(new)
```

	Car_Name	Year	Selling_Price	Present_Price	Driven_kms	Fuel_Type	Selling_type	Transmission	
0	ritz	2015	4.0	5.5	25000	Petrol	Dealer	Manual	

```
new_processed = transformer.transform(new)
predicted_count = model.predict(new_processed)
print("Predicted number of owners (0 for first owner, 1
```

```
Predicted number of owners (0 for first owner, 1 for second, etc.): -0.06946717069174468
```